Final Project: Electrical Job Management System

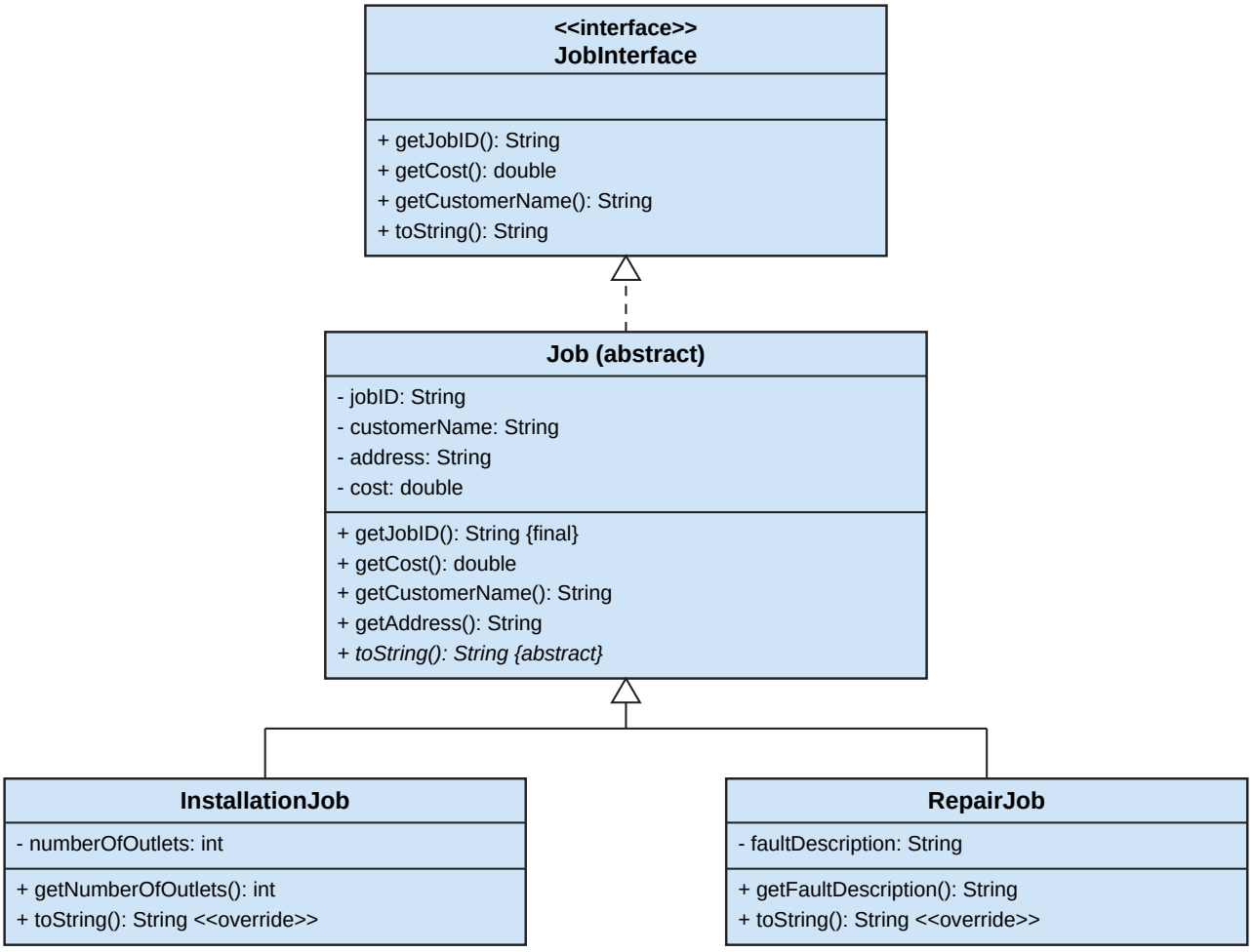
977-210 Object-oriented Programming

Name: Nakanya Payuksa **Student ID:** 6930613009

1. Target System

My father is an electrician. This program manages his jobs. There are two kinds of jobs: **Installation Jobs** (new outlets or wiring) and **Repair Jobs** (fixing a fault). The user can add jobs, view all jobs, and search for jobs.

2. UML Class Diagram



Dashed arrow = implements | Solid arrow = extends | All jobs are stored in one shared `List<JobInterface>` (an ArrayList).

> **Change from the proposal:** I added `getAddress()` to `Job` so that `toString()` can show the address. Everything else is the same as the proposal.

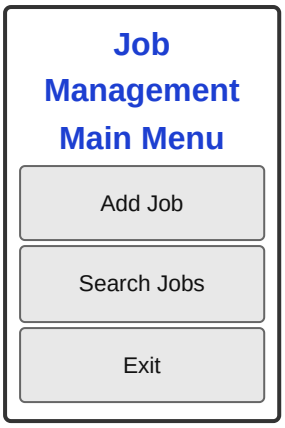
3. How the Project Meets the Requirements

Requirement	Where it is in my code
One interface	`JobInterface`
Abstract class that implements the interface	`Job implements JobInterface`
A `final` method	`public final String getJobID()` in `Job`
Two concrete subclasses	`InstallationJob` and `RepairJob`, both `extends Job`
Unique attribute in each subclass	`numberOfOutlets` (InstallationJob), `faultDescription` (RepairJob)
Unique method in each subclass	`getNumberOfOutlets()`, `getFaultDescription()`
Encapsulation	All attributes are `private`. Other classes use public getters. The shared list in `Database` is also `private`.
Polymorphic ArrayList	`List<JobInterface> jobs = new ArrayList<>()` in `Database`. It stores both subclass types.
3 Swing pages	`MainPage`, `DataEntryPage`, `SearchPage`
Exception handling	`try-catch` for `NumberFormatException` in `DataEntryPage` and `SearchPage` shows a message dialog.

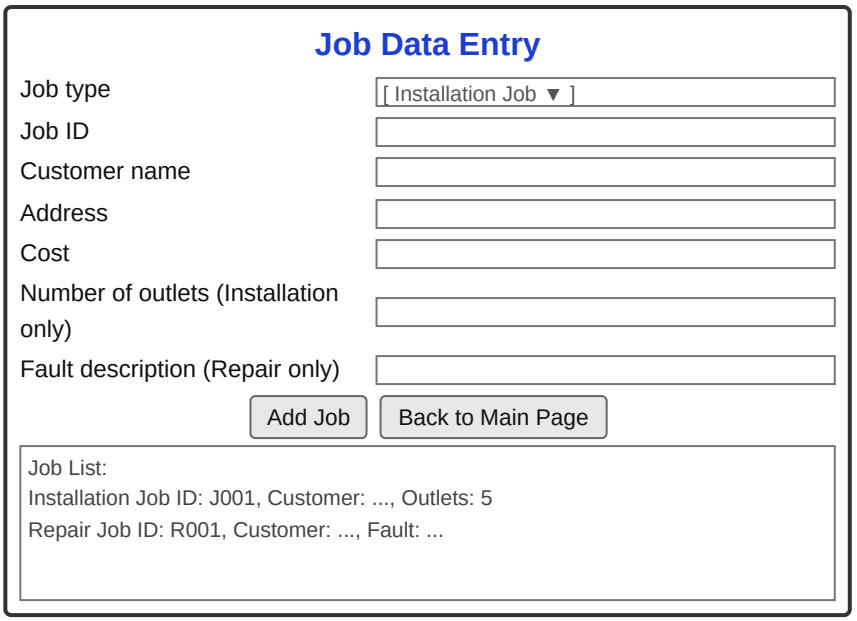
4. GUI Design

Page	Purpose	Main Swing components
Main	Go to the other pages.	JLabel (title), JButtons: Add Job, Search Jobs, Exit
Data Entry	Add a job and show all jobs in the shared list.	JComboBox (job type), JTextFields (ID, customer, address, cost, outlets, fault), JButtons (Add Job, Back), JTextArea (all jobs)
Search	Find jobs by a chosen criterion.	JComboBox (criterion), JTextField (value), one Search JButton, Back JButton, JTextArea (results)

Page sketches



Main Page



Data Entry Page



Search Page

How the Data Entry page supports both subclasses

The user chooses **Installation Job** or **Repair Job** in the combo box. If Installation is chosen, the program reads the number of outlets and creates an `InstallationJob`. If Repair is chosen, it reads the fault description and creates a `RepairJob`. Both objects are added to the same shared list by `Database.addJob(job)`. The text area then shows every job in the list by calling `toString()`.

Search criteria

Criterion	Applies to
Cost >= value	Both (common)
Customer name contains text	Both (common)
Number of outlets >= value	InstallationJob only (unique)
Fault description contains text	RepairJob only (unique)

For a criterion that belongs to only one subclass, the program uses `instanceof` first. Objects of the other subclass are skipped, so no error happens. All four criteria use one Search button and one loop.

Invalid input

The program uses `try-catch`. If the user types text instead of a number (for example "abc" in Cost), a `NumberFormatException` is caught and a message dialog is shown. The program does not crash.

5. Test Cases

#	Action	Expected result
1	Data Entry: type = Installation Job, ID J001, customer Somchai, address 12 Main Rd, cost 1500, outlets 5, click Add Job	Dialog "Add job successfully." The job appears in Job List.
2	Data Entry: type = Repair Job, ID R001, customer Malee, address 99 Lake St, cost 800, fault "Short circuit", click Add Job	Dialog "Add job successfully." Both jobs are now in the list.
3	Data Entry: cost = abc	Dialog "Wrong input. Cost and outlets must be numbers." No job is added.
4	Search: Cost >= 1000	Only J001 is shown.
5	Search: Customer name contains "mal"	Only R001 is shown (not case-sensitive).
6	Search: Number of outlets >= 3	Only J001 is shown. RepairJob objects are skipped safely.
7	Search: Fault description contains "short"	Only R001 is shown. InstallationJob objects are skipped safely.
8	Search: Number of outlets >= x	Dialog "Wrong input. This criterion needs a number."
9	Search: Fault description contains "zzz"	Results show "Not Found".

6. Source Code

Put all files in the same folder (default package). Compile with `javac *.java` and run `java MainPage`.

JobInterface.java

The interface. It declares the behaviour that every job must have.

```
public interface JobInterface {
    String getJobID();
    double getCost();
    String getCustomerName();
    String toString();
}
```

Job.java

The abstract superclass. It implements the interface and holds the data shared by all jobs. `getJobID()` is **final**.

```
public abstract class Job implements JobInterface {
    private String jobID;
    private String customerName;
    private String address;
    private double cost;

    public Job(String jobID, String customerName, String address, double cost) {
        this.jobID = jobID;
        this.customerName = customerName;
        this.address = address;
        this.cost = cost;
    }

    @Override
    public final String getJobID() {       // final: subclasses cannot override
```

```
        return jobID;
    }

    @Override
    public double getCost() {
        return cost;
    }

    @Override
    public String getCustomerName() {
        return customerName;
    }

    public String getAddress() {
        return address;
    }

    @Override
    public abstract String toString();     // each subclass writes its own
}
```

InstallationJob.java

Concrete subclass 1. Unique attribute: `numberOfOutlets`. Unique method: `getNumberOfOutlets()`.

```
public class InstallationJob extends Job {
    private int numberOfOutlets;           // unique attribute

    public InstallationJob(String jobID, String customerName, String address,
                           double cost, int numberOfOutlets) {
        super(jobID, customerName, address, cost);
        this.numberOfOutlets = numberOfOutlets;
    }

    public int getNumberOfOutlets() {      // unique method
        return numberOfOutlets;
    }

    @Override
    public String toString() {
        return "Installation | ID: " + getJobID() + " | Customer: " + getCustomerName()
                + " | Address: " + getAddress() + " | Cost: " + getCost()
                + " | Outlets: " + numberOfOutlets + "\n";
    }
}
```

RepairJob.java

Concrete subclass 2. Unique attribute: `faultDescription`. Unique method: `getFaultDescription()`.

```
public class RepairJob extends Job {
    private String faultDescription;       // unique attribute

    public RepairJob(String jobID, String customerName, String address,
                     double cost, String faultDescription) {
        super(jobID, customerName, address, cost);
        this.faultDescription = faultDescription;
    }

    public String getFaultDescription() {  // unique method
        return faultDescription;
    }

    @Override
    public String toString() {
        return "Repair | ID: " + getJobID() + " | Customer: " + getCustomerName()
                + " | Address: " + getAddress() + " | Cost: " + getCost()
```

```
                + " | Fault: " + faultDescription + "\n";
    }
}
```

Database.java

Holds the ONE shared `ArrayList<JobInterface>` used by every page.

```
import java.util.ArrayList;
import java.util.List;

public class Database {
    // ONE shared list for both job types (declared with the interface type)
    private static final List<JobInterface> jobs = new ArrayList<>();

    public static void addJob(JobInterface job) {
        jobs.add(job);
    }

    public static List<JobInterface> getJobs() {
        return jobs;
    }
}
```

MainPage.java

Landing page. Opens the Data Entry page and the Search page, or exits.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MainPage extends JFrame implements ActionListener {

    public MainPage() {
        super("Electrical Job Management System");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(300, 250);

        JButton addBtn = new JButton("Add Job");
        JButton searchBtn = new JButton("Search Jobs");
        JButton exitBtn = new JButton("Exit");
        addBtn.addActionListener(this);
        searchBtn.addActionListener(this);
        exitBtn.addActionListener(this);

        JPanel panel = new JPanel(new GridLayout(3, 1));
        panel.add(addBtn);
        panel.add(searchBtn);
        panel.add(exitBtn);

        add(new JLabel("Job Management Main Menu", SwingConstants.CENTER), BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();
        if (command.equals("Add Job")) {
            new DataEntryPage();
        } else if (command.equals("Search Jobs")) {
            new SearchPage();
        } else if (command.equals("Exit")) {
            System.exit(0);
        }
    }
```

```
    public static void main(String[] args) {
        new MainPage();
    }
}
```

DataEntryPage.java

Creates an `InstallationJob` or a `RepairJob`, adds it to the shared list and shows all jobs.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class DataEntryPage extends JFrame implements ActionListener {
    private JComboBox<String> jobTypeBox = new JComboBox<>(new String[]{"Installation Job", "Re
pair Job"});
    private JTextField idTxt = new JTextField();
    private JTextField customerTxt = new JTextField();
    private JTextField addressTxt = new JTextField();
    private JTextField costTxt = new JTextField();
    private JTextField outletsTxt = new JTextField();
    private JTextField faultTxt = new JTextField();
    private JTextArea jobListArea = new JTextArea(10, 40);

    public DataEntryPage() {
        super("Job Data Entry");
        setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
        setSize(600, 550);

        JPanel form = new JPanel(new GridLayout(7, 2));
        form.add(new JLabel("Job type"));
        form.add(jobTypeBox);
        form.add(new JLabel("Job ID"));
        form.add(idTxt);
        form.add(new JLabel("Customer name"));
        form.add(customerTxt);
        form.add(new JLabel("Address"));
        form.add(addressTxt);
        form.add(new JLabel("Cost"));
        form.add(costTxt);
        form.add(new JLabel("Outlets (Installation only)"));
        form.add(outletsTxt);
        form.add(new JLabel("Fault (Repair only)"));
        form.add(faultTxt);

        JButton addBtn = new JButton("Add Job");
        JButton backBtn = new JButton("Back to Main Page");
        addBtn.addActionListener(this);
        backBtn.addActionListener(this);
        JPanel buttons = new JPanel();
        buttons.add(addBtn);
        buttons.add(backBtn);

        JPanel top = new JPanel(new BorderLayout());
        top.add(form, BorderLayout.CENTER);
        top.add(buttons, BorderLayout.SOUTH);

        jobListArea.setEditable(false);
        for (JobInterface j : Database.getJobs()) {   // show jobs already stored
            jobListArea.append(j.toString());
        }

        add(new JLabel("Job Data Entry", SwingConstants.CENTER), BorderLayout.NORTH);
        add(top, BorderLayout.CENTER);
        add(new JScrollPane(jobListArea), BorderLayout.SOUTH);
        setVisible(true);
    }
```

```
    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();
        if (command.equals("Add Job")) {
            try {
                String id = idTxt.getText();
                String customer = customerTxt.getText();
                String address = addressTxt.getText();
                double cost = Double.parseDouble(costTxt.getText());   // may throw

                JobInterface job;
                if (jobTypeBox.getSelectedItem().equals("Installation Job")) {
                    int outlets = Integer.parseInt(outletsTxt.getText());   // may throw
                    job = new InstallationJob(id, customer, address, cost, outlets);
                } else {
                    job = new RepairJob(id, customer, address, cost, faultTxt.getText());
                }

                Database.addJob(job);
                jobListArea.append(job.toString());
                JOptionPane.showMessageDialog(this, "Add job successfully.");
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Wrong input. Cost and outlets must be numb
ers.");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Some error occurred.\n" + ex);
            }
        } else if (command.equals("Back to Main Page")) {
            dispose();
        }
    }
}
```

SearchPage.java

One Search button, four criteria. Shows matching jobs.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SearchPage extends JFrame implements ActionListener {
    private JComboBox<String> criterionBox = new JComboBox<>(new String[]{
            "Cost >=", "Customer name contains", "Number of outlets >=", "Fault description con
tains"});
    private JTextField valueTxt = new JTextField(15);
    private JTextArea resultsArea = new JTextArea(15, 50);

    public SearchPage() {
        super("Job Search");
        setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
        setSize(600, 450);

        JButton searchBtn = new JButton("Search");
        JButton backBtn = new JButton("Back to Main Page");
        searchBtn.addActionListener(this);
        backBtn.addActionListener(this);

        JPanel inputPanel = new JPanel();
        inputPanel.add(new JLabel("Criterion:"));
        inputPanel.add(criterionBox);
        inputPanel.add(new JLabel("Value:"));
        inputPanel.add(valueTxt);
        inputPanel.add(searchBtn);
        inputPanel.add(backBtn);

        resultsArea.setEditable(false);
```

```
        add(new JLabel("Job Search", SwingConstants.CENTER), BorderLayout.NORTH);
        add(inputPanel, BorderLayout.CENTER);
        add(new JScrollPane(resultsArea), BorderLayout.SOUTH);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();
        if (command.equals("Search")) {
            try {
                String criterion = (String) criterionBox.getSelectedItem();
                String value = valueTxt.getText().toLowerCase();
                double number = 0;
                if (criterion.equals("Cost >=") || criterion.equals("Number of outlets >=")) {
                    number = Double.parseDouble(value);          // may throw
                }

                resultsArea.setText("Results:\n");
                boolean found = false;
                for (JobInterface j : Database.getJobs()) {
                    boolean match = false;
                    if (criterion.equals("Cost >=")) {
                        match = j.getCost() >= number;                        // both types
                    } else if (criterion.equals("Customer name contains")) {
                        match = j.getCustomerName().toLowerCase().contains(value);   // both ty
pes
                    } else if (criterion.equals("Number of outlets >=")) {
                        if (j instanceof InstallationJob) {                   // only this type
                            match = ((InstallationJob) j).getNumberOfOutlets() >= number;
                        }
                    } else {
                        if (j instanceof RepairJob) {                         // only this type
                            match = ((RepairJob) j).getFaultDescription().toLowerCase().contain
s(value);

                        }
                    }
                    if (match) {
                        resultsArea.append(j.toString());
                        found = true;
                    }
                }
                if (!found) {
                    resultsArea.append("Not Found\n");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Wrong input. This criterion needs a numbe
r.");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Some error occurred.\n" + ex);
            }
        } else if (command.equals("Back to Main Page")) {
            dispose();
        }
    }
}
```